

Lab7 File System

2026-6-7 王天择

Lab内容

- **Task 1** (*increase file size*). 通过修改 *xv6 inode* 实现来支持更大的文件。
 - 首先了解 *xv6 inode* 大小限制的原因, 以及 *bmap()* 读写文件时 *block* 的分配方式。
 - 了解通过修改直接块和间接块的 *inode* 结构来增大最大可用 *file size* 的方法。
- **Task 2** (*symbolic links*). 加入一种特殊的文件 —— 符号链接文件。
 - 符号链接文件的文件内容是一个文件路径, 打开该文件相当于打开指向的文件。
 - 添加和修改系统调用, 来创建和 (打开时) 解析符号链接文件。
 - 由于符号链接文件指向的也可能是符号链接文件, 需要额外考虑递归解析和处理。

Review 1: xv6 的磁盘组织结构

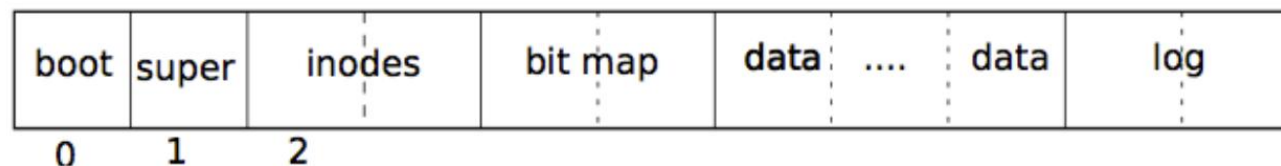


Figure 6-2. Structure of the xv6 file system. The header `fs.h` (3650) contains constants and data structures describing the exact layout of the file system.

Log

存储磁盘操作的日志，主要用于实现系统的崩溃恢复

启动扇区

读入 xv6 内核文件，将控制权转交给 kernel 代码，从而运行 OS

超级块

包含文件系统的元信息，例如总块数、数据块块数、inode 节点数等

inodes

存放 inode 索引节点，每一个节点对应一个文件的元信息 (e.g., 大小)

空闲块位图

标记哪个 data 块是空闲的

datas

实际存储文件数据的空间，保存文件和目录的内容

Review 2: xv6 的文件系统多级结构

更清晰的代码结构，容易理解

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache ←
Disk

块缓冲层

- 任务一：同步对磁盘的访问，使得对于每一个块，同一时间只有一份拷贝放在内存中并且只有一个内核线程使用这份拷贝。
- 任务二：缓存常用的块以提升性能。
- 块缓冲提供的主要接口是 `bread` 和 `bwrite`；前者从磁盘中取出一块放入缓冲区，后者把缓冲区中的一块写到磁盘上正确的地方。当内核处理完一个缓冲块之后，需要调用 `brelease` 释放它。
- 块缓冲仅允许最多一个内核线程引用它，以此来同步对磁盘的访问，如果一个内核线程引用了一个缓冲块，但还没有释放它，那么其他调用 `bread` 的进程就会阻塞。文件系统的更高几层正是依赖块缓冲层的同步机制来保证其正确性。
- 块缓冲有固定数量的缓冲区，这意味着如果文件系统请求一个不在缓冲中的块，必须换出一个已经使用的缓冲区 (`LRU`)。

Review 2: xv6 的文件系统多级结构

File descriptor
Pathname
Directory
Inode
Logging 
Buffer cache
Disk

日志层

- 大多数的文件系统都涉及到对磁盘多次的写操作，如果在写操作的过程当中系统崩溃了，就会使得磁盘上的文件系统处于不一致的状态中。
- **xv6** 通过简单的日志系统来解决文件操作过程当中崩溃所导致的问题。一个系统调用并不直接导致对磁盘上文件系统的写操作，相反，他会把一个对磁盘写操作的描述包装成一个日志写在磁盘中。当系统调用把所有的写操作都写入了日志，它就会写一个特殊的提交记录到磁盘上，代表一次完整的操作。从那时起，系统调用就会把日志中的数据写入磁盘文件系统的数据结构中。在那些写操作都成功完成后，系统调用就会删除磁盘上的日志文件。

Review 2: xv6 的文件系统多级结构

File descriptor
Pathname
Directory
Inode 
Logging
Buffer cache
Disk

Inode 层

- 所有的磁盘上的 i 节点都被打包在一个称为 i 节点块的连续区域中。每一个 i 节点的大小都是一样的，所以对于一个给定的数字 n ，很容易找到磁盘上对应的 i 节点。事实上这个给定的数字就是操作系统中 i 节点的编号。
- 磁盘上的 i 节点由结构体 `dinode` (3676) 定义。type 域用来区分文件、目录和特殊文件的 i 节点。如果 type 是 0 的话就意味着这是一个空闲的 i 节点。
- 内核在内存中维护活动的 i 节点。结构体 `inode` (3762) 是磁盘中的结构体 `dinode` 在内存中的拷贝。内核只会在有 `C` 指针指向一个 i 节点的时候才会把这个 i 节点保存在内存中。ref 域用于统计有多少个 `C` 指针指向它。如果 ref 变为 0，内核就会丢掉这个 i 节点。iget 和 iput 两个函数申请和释放 i 节点指针，修改引用计数。 i 节点指针可能从文件描述符产生，从当前工作目录产生，也有可能从一些内核代码如 `exec` 中产生。

Review 2: xv6 的文件系统多级结构

File descriptor	←
Pathname	←
Directory	←
Inode	
Logging	
Buffer cache	
Disk	

目录层

- 在xv6中，目录的实现和文件的实现过程很像。目录的 i 节点的类型 `T_DIR`，它的数据是一系列的目录条目。每个条目是一个 `struct dirent` (3700) 结构体，包含一个名字和一个 i 节点编号。这个名字最多有 `DIRSIZ` (14) 个字符；如果比较短，它将以 `NUL` (0) 作为结尾字符。 i 节点编号是 0 的条目都是可用的。

路径层

- 从一个文件路径（如 `/bin/bash`）得到该文件的 `inode`。通过反复调用 `dirlookup()` 实现。

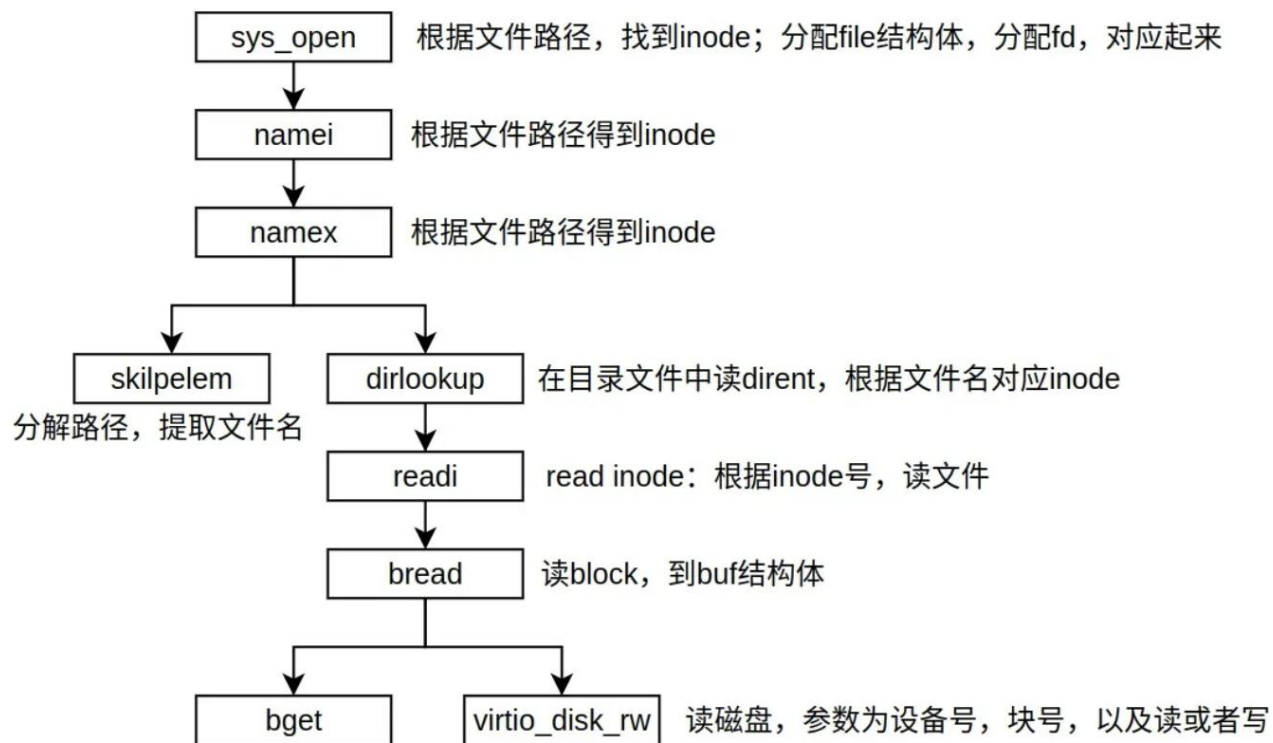
文件描述符层

- 用来实现 `unix` 中的文件抽象。`unix` 将设备、管道以及实际的文件都抽象到一个统一的文件接口上。

Review 2: xv6 的文件系统多级结构

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

打开文件全过程



Task 1: 通过修改 *xv6* *inode* 实现来支持大文件

1. 实验原理:

➤ 当前 xv6 实现中 inode 结构:

Direct Block

直接地址, 指向 1 个 data block 的地址, 更快

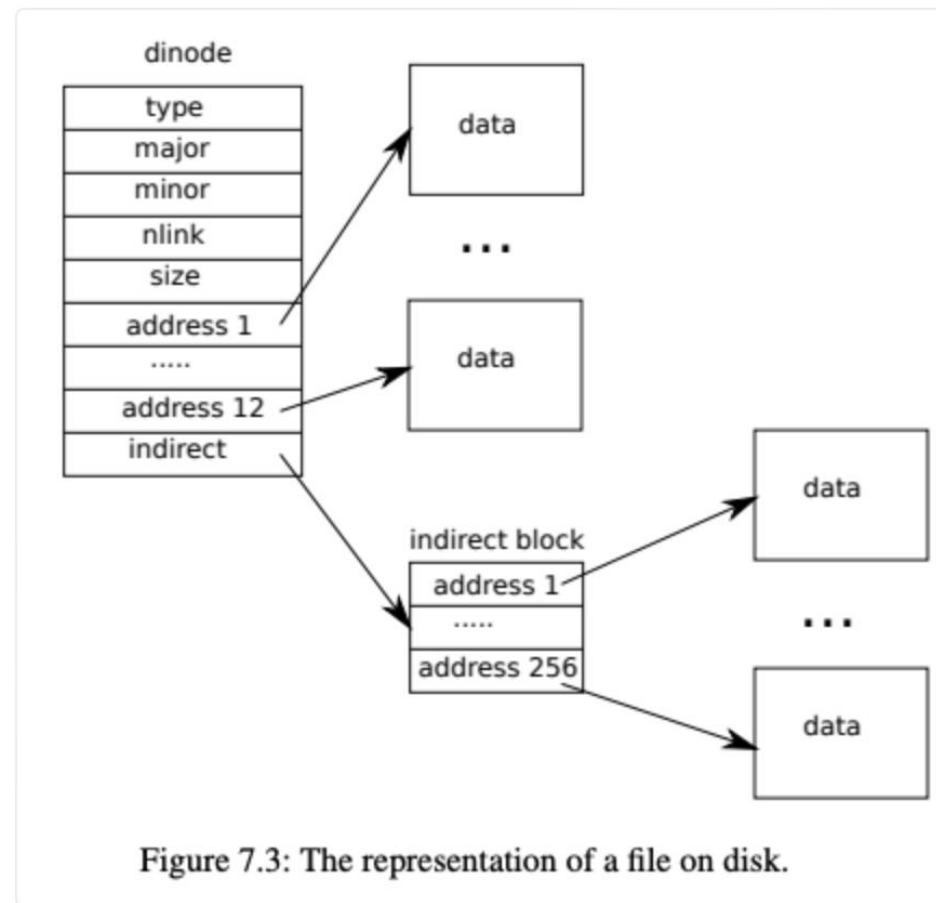
Indirect Block

一级块表, 包括 256 个 data block 的地址, 更大

$$12 \times 1 + 1 \times 256$$

$$= 268 \text{ blocks}$$

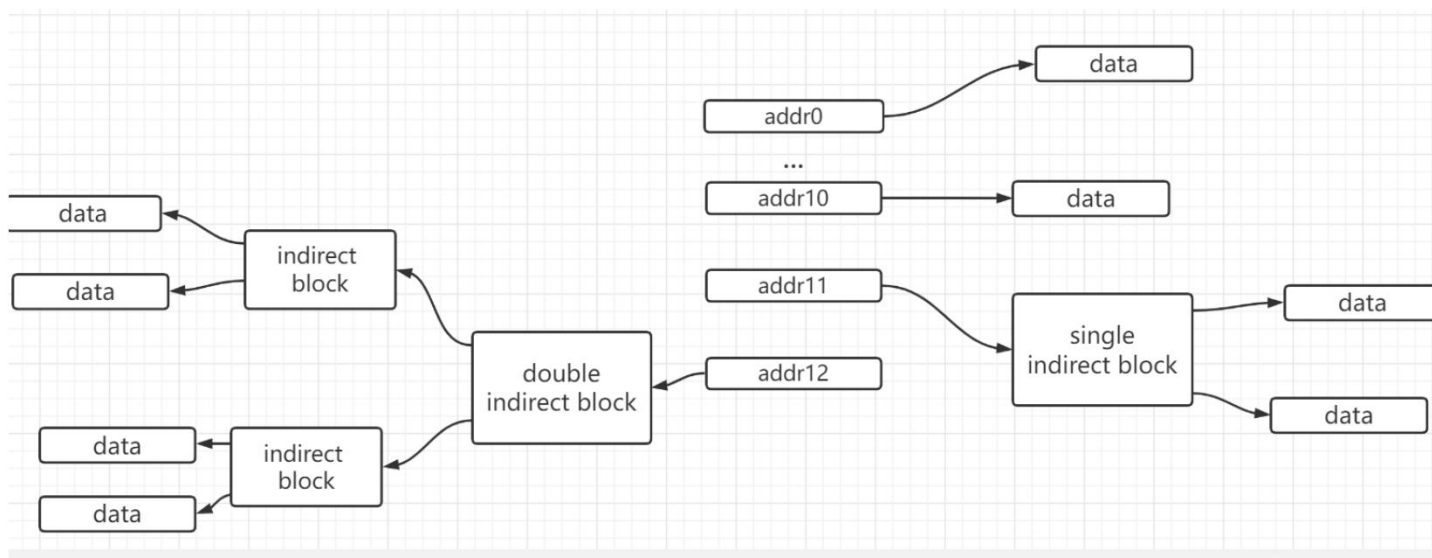
$$= 268 \times 1024 \text{ bytes} = 268 \text{ KB}$$



Task 1: 通过修改 *xv6* *inode* 实现来支持大文件

1. 实验原理:

➤ 改进后的 xv6 inode 结构:



Direct Block

直接地址, 指向 1 个 data block 的地址, 更快

$$11 \times 1$$

+

Indirect Block

一级块表, 包括 256 个 data block 的地址, 更大

$$1 \times 256$$

+

Double Indirect Block

二级块表, 包括 256 个 Indirect blocks, 更更大

$$1 \times 256 \times 256$$

||

65803 blocks

246x Larger!

Task 1: 通过修改 *xv6 inode* 实现来支持大文件

2. 实验提示:

- 确保理解 *bmap()*。写出 *ip->addrs[]*、间接块、双向块及其指向的单向块和数据块之间的关系图。
- 想想如何用逻辑块编号来索引双向块及其指向的间接块。
- 如果更改 *NDIRECT* 的定义，很可能需要更改 *file.h* 中 *struct inode* 中 *addrs[]* 的声明。确保 *struct inode* 和 *struct dinode* 的 *addrs[]* 数组中的元素数量相同。
- 如果更改了 *NDIRECT* 的定义，请确保创建了新的 *fs.img*，因为 *mkfs* 使用 *NDIRECT* 来构建文件系统。
- 运行 *usertests* 所需的时间比以前的实验要长，因为本实验中的 *FSSIZE* 较大，大文件也较大。

Task 1: 通过修改 *xv6 inode* 实现来支持大文件

3. 实验步骤:

这个 lab 的目的就是将一个直接数据块号替换成一个两层间接数据块号，即指向一个包含间接数据块号的数据块。主要的代码就是修改 *bmap* 函数 (*kernel/fs.c*)，它会将指定 *inode* 的第 *bn* 个数据块（相对位置）映射到磁盘的数据块号。

```
1 $ bigfile
2 .....
3 wrote 65803 blocks
4 done; ok
5 $ usertests -q
6 ...
7 ALL TESTS PASSED
8 $
```

```
uint indirect_idx, final_offset;
struct buf *bp2;
if (bn < NDOUBLEINDIRECT) {
    // Load double-indirect block, allocating if necessary
```

Insert here

```
}
```

```
panic("bmap: out of range");
```

Task 2: 加入符号链接文件

1. 实验要求:

本练习将为 *xv6* 添加符号链接。符号链接（或软链接）通过路径名指向被链接的文件；打开符号链接时，内核会根据链接指向被引用的文件。符号链接类似于硬链接，但硬链接仅限于指向同一磁盘上的文件，而符号链接可以跨越磁盘设备。

实现 `symlink(char target, char path)` 系统调用，在 *linkpath* 处创建一个新的符号链接，指向以 *target* 命名的文件。

Task 2: 加入符号链接文件

2. 实验提示:

- 为 *symlink* 创建一个新的系统调用编号, 在 *user/usys.pl* 和 *user/user.h* 中添加一个条目, 并在 *kernel/sysfile.c* 中实现一个空的 *sys_symlink*。
- 在 *kernel/fcntl.h* 中添加一个新标志 (*O_NOFOLLOW*), 可与 *open* 系统调用一起使用。
- 执行 *symlink(target, path)* 系统调用, 在 *path* 处创建指向 *target* 的新符号链接。
- 修改 *open* 系统调用, 以处理路径指向符号链接的情况。如果文件不存在, *open* 必须失败。
- 如果链接文件也是一个符号链接, 则必须递归地跟踪它, 直到找到一个非链接文件。如果链接形成循环, 则必须返回错误代码。

Task 2: 加入符号链接文件

3. 实验步骤:

- 首先新增 *syslink* 这个系统调用 (*kernel/sysfile.c*), 这个系统调用会创建一个类型为 *T_SYMLINK* 的文件, 并把 *target* 的文件路径写到它的 *datablock* 中。

```
uint64  
sys_symlink(void)  
{
```



Insert here

```
}
```

Task 2: 加入符号链接文件

3. 实验步骤:

- 另外需要修改 `open` 这个系统调用, 使得操作系统能够正确处理 `symlink` 类型的文件。
- 注意如果软链接文件链接到了另一个软链接文件, 系统需要递归地查找直到找到第一个非软链接的文件 (对于成环的情况, 需要返回-1)。
- 如果用户在打开文件时添加了 `O_NOFOLLOW` 模式, 那么系统需要像处理正常文件一样将软链接文件打开。

```
1 $ symlinktest
2 Start: test symlinks
3 test symlinks: ok
4 Start: test concurrent symlinks
5 test concurrent symlinks: ok
6 $ usertests -q
7 ...
8 ALL TESTS PASSED
9 $
```

```
if (ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)) {
```

Insert here

```
}
```